Reproducing "Less is More: Recursive Reasoning with Tiny Networks" (TRM): A Reduced-Scale Study on Sudoku

prince kumar sahu 24CS3038
Department of Computer Science and Engineering
Course 24CS3038 | Paper ID DL-064 | RGIPT, 24cs3038@rgipt.ac.in

***Abstract*—Tiny Recursive Model (TRM) solves hard puzzles with a single 2-layer network that repeatedly refines a latent "reasoning" state z and an answer y, trained with deep supervision. We re-implement TRM from scratch in PyTorch and reproduce its Sudoku experiment at reduced scale: 4×4 Sudoku, 100 training and 120 test puzzles, a 268k-parameter model, and no data augmentation. The model reaches $86.3 \pm 3.4\%$ exact-solution accuracy (3 seeds) and corrects its own mistakes across supervision steps. We ablate recursion depth (n, T), supervision steps (N_{sup}), the 1-step gradient approximation, and EMA. Only some of the paper's claims transfer: deep supervision and EMA help in the expected direction, but the large gain from full back-propagation and from deeper recursion does not appear at this scale. Apart from very few supervision steps, ablation gaps are mostly within seed noise.**

I. Problem and Motivation

Large language models generate answers token by token, so one wrong token can invalidate a whole answer. Chain-of-thought and test-time compute reduce this but are expensive and still fail on puzzle-style tasks such as Sudoku-Extreme, Maze-Hard and ARC-AGI [1]. The Hierarchical Reasoning Model (HRM) [2] showed that a small supervised network (27M parameters, about 1000 examples) can solve such tasks by recursing, but its design leans on fixed-point theory and biological arguments, and its gains were later traced mostly to deep supervision rather than hierarchy [3].

TRM [1] strips HRM down: *one* 2-layer network, *two* states (y = current answer, z = latent reasoning), full back-propagation through one recursion block, and a cheaper halting rule. With 5–7M parameters it reports 87.4% on Sudoku-Extreme (HRM: 55.0%). The central question we test is whether this recipe—tiny network + recursion + deep supervision—also works in a small-compute setting, and which of its ingredients actually matter.

II. Method

A. Latent recursion

Let $x \in \mathbb{R}^{L\times D}$ be the embedded puzzle (L=16 cells), y the embedded current answer and z the latent state, with y and z initialised to zero. A single network f_θ plays both roles. The role is implied by the input: with x present it updates the latent, without x it updates the answer:

$$z \leftarrow f_\theta(x + y + z) \quad \text{(repeated } n \text{ times)}, \tag{1}$$

$$y \leftarrow f_\theta(y + z). \tag{2}$$

We write one execution of (1)–(2) as $(y', z') = \mathcal{R}_\theta(x, y, z)$; it costs n+1 applications of f_θ.

B. Deep recursion and gradient

One *deep recursion* applies $\mathcal{R}_\theta$ T times, but only the last application is differentiated:

$$(y, z) \leftarrow \mathrm{sg}\big(\mathcal{R}_\theta^{T-1}(x, y, z)\big), \qquad (\hat{y}, z) \leftarrow \mathcal{R}_\theta(x, y, z). \tag{3}$$

Here sg is stop-gradient. Unlike HRM, which back-propagates through only the last two evaluations under a fixed-point assumption, TRM back-propagates through all n+1 evaluations of the final block, so no fixed-point theorem is needed. Memory scales with n+1, not with $T(n$+1).

Gradient (board derivation). Let $h_0 = (x, y, z)$ be the (detached) input to the graded block and let h_k be the state after the k-th application of f_θ, $k = 1, \ldots, n$+1 (the first n update z, the last updates y). Since θ is shared across all n+1 applications,

$$\nabla_\theta \mathcal{L} = \sum_{k=1}^{n+1} \Big(\frac{\partial \mathcal{L}}{\partial h_k}\Big)^\top \frac{\partial f_\theta}{\partial \theta}\Big|_{h_{k-1}}, \quad \frac{\partial \mathcal{L}}{\partial h_k} = \frac{\partial \mathcal{L}}{\partial h_{k+1}} \frac{\partial h_{k+1}}{\partial h_k}, \tag{4}$$

where the recursion on $\partial\mathcal{L}/\partial h_k$ starts at $h_{n+1} = y$ and stops at h_0 because (y, z) from earlier blocks and earlier supervision steps are constants. The 1-step approximation of HRM keeps only the last one or two terms of (4), which is exact only if the state is at a fixed point; TRM keeps all n+1 terms.

C. Heads, loss and deep supervision

The answer head and a halting head are

$$\hat{p} = \mathrm{softmax}(W_o\, y), \qquad q = \sigma\big(w_q^\top \tfrac{1}{L} \textstyle\sum_\ell y_\ell\big). \tag{5}$$

For a puzzle with solution $y^\star$ the per-step loss is

$$\mathcal{L} = \tfrac{1}{L} \textstyle\sum_{\ell=1}^{L} \mathrm{CE}(\hat{p}_\ell, y_\ell^\star) + \mathrm{BCE}\big(q,\ \mathbf{1}[\arg\max \hat{p} = y^\star]\big). \tag{6}$$

Deep supervision repeats this for $s = 1, \ldots, N_{sup}$ on the same batch: (y_s, z_s) from step s are detached and become the starting point of step s+1, and *every* step performs its own optimiser update. The model therefore learns to map any (y, z) to a better (y, z), which lets it fix its own mistakes at inference by iterating the same N_{sup} steps. The effective depth per supervision step is $T(n$+1$)\,\ell$ layers (ℓ=2), i.e. $3 \cdot 7 \cdot 2 = 42$ for n=6, T=3, and 672 over N_{sup}=16 steps. Weights used for evaluation are an exponential moving average $\theta_{\text{ema}} \leftarrow 0.999\,\theta_{\text{ema}} + 0.001\,\theta$.

D. Attention-free block

Following the paper's finding that an MLP over the sequence axis beats self-attention on small fixed grids (Sudoku), each of the 2 layers is, for $h \in \mathbb{R}^{L\times D}$,

$$\tilde{h} = \mathrm{LN}\big(h + [W_2\, \phi(W_1 h^\top)]^\top\big), \tag{7}$$

$$h' = \mathrm{LN}\big(\tilde{h} + V_2\, \phi(V_1 \tilde{h})\big), \tag{8}$$

Table I
SETUP: PAPER VS. THIS REPRODUCTION

	Paper (Sudoku-Extreme)	Ours
Grid / train / test	9×9 / 1K / 423K	4×4 / 100 / 120
Augmentation	1000× shuffles	none
Hidden size D	512	128
Parameters	5M (MLP variant)	0.27M
Block	SwiGLU + RMSNorm	GELU + LayerNorm
Optimiser	AdamW, 10^{-4}, wd 1.0	AdamW, 10^{-3}, wd 0.01
Batch / epochs	768 / 60K	64 / 100
n, T, N_{sup}	6, 3, 16	6, 3, 16
Halting (ACT)	learned, early stop	head trained, *not used*

```
def R(x, y, z, n):        # latent recursion
    for _ in range(n):
        z = f(x + y + z)
    y = f(y + z)
    return y, z

for puz, sol in batches:
    y, z = 0, 0
    for s in range(N_sup):    # deep supervision
        x = embed(puz)
        with no_grad():
            for _ in range(T - 1):
                y, z = R(x, y, z, n)
        y, z = R(x, y, z, n) # graded block
        loss = CE(head(y), sol) \
             + BCE(qhead(mean(y)), ok)
        loss.backward()
        AdamW.step(); EMA.update()
        y, z = y.detach(), z.detach()
```

Figure 1. Pseudocode of our training loop (core of the notebook).

with $W_1 \in \mathbb{R}^{2L\times L}$, $V_1 \in \mathbb{R}^{4D\times D}$, ϕ =GELU. This is the token-mixing idea of MLP-Mixer [4].

III. IMPLEMENTATION DETAILS

Code. The core module (`TRM`, `latent_recursion`, `deep_recursion`, EMA, deep-supervision loop) is written by us in PyTorch following the pseudocode of Fig. 3 in [1]; everything runs in one Colab notebook, and every run is saved to JSON for resumption.

Data. Because the paper's Sudoku-Extreme set is 9×9 and needs hours of GPU time, we generate 4×4 Sudoku: a random full grid is built by backtracking, then cells are removed while the solution stays unique, down to 4 clues (the minimum for a 4×4 grid). We use 100 training and 120 test puzzles, de-duplicated by *solution* so no test solution grid appears in training (note there are only 288 distinct 4×4 grids). Tokens are $\{0,\dots,4\}$ with 0 = blank, L=16.

Model and training. D=128, ℓ=2 blocks, one shared network, 268,006 parameters. Defaults: n=6, T=3, N_{sup}=16, 100 epochs, batch 64 (2 iterations per epoch), AdamW with learning rate 10^{-3} and weight decay 0.01, EMA 0.999, no augmentation. The input is re-embedded at each supervision step. The metric is *exact-match*: a puzzle counts only if all 16 cells are correct after N_{sup} steps. A run takes 0.1–1.6 minutes; all ablations use seeds 0, 1, 2.

Evaluation protocol. After training we load the EMA weights, start from y=z=0, run N_{sup} supervision steps without gradients and take the arg-max of the answer head at the last step. Training and test puzzles come from generator seeds 0 and 1. The sample standard deviation over three training seeds (0,1,2) is reported; with n=120 test puzzles and accuracy ≈ 0.86, one run has a binomial standard error of $\sqrt{0.86\cdot 0.14/120} \approx 3.2$ points.

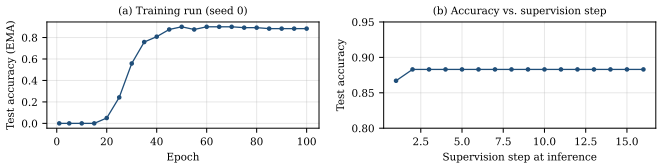


Figure 2. (a) Test accuracy of the EMA model during training (seed 0, 100 epochs). (b) Accuracy of the trained model as the number of supervision steps at inference grows.

Deviations. Table I lists what changed. Notably, our halting head is trained (BCE term) but we always run all N_{sup} steps, so there is no ACT-based speed-up. Numbers are therefore *not directly comparable* with the paper; we compare trends.

Reproducibility. Code, README with setup and run instructions, seeds and the results table, and a runnable Colab notebook are in the repository at [GitHub URL]. Results are stored per run as JSON (name, n, T, N_{sup}, seed, accuracy per supervision step, minutes).

IV. RESULTS VS. PAPER

A. Main result

The default configuration reaches $86.3 \pm 3.4\%$ exact accuracy (mean ± std over 3 seeds; individual runs 83.2, 85.8, 90.0). For reference, the paper reports 87.4% for the same recipe on the much harder 9×9 task. The two numbers measure different tasks and must not be read as a like-for-like match; what carries over is that a tiny network with recursion and deep supervision learns the task from very few examples.

Fig. 2(a) shows the training dynamics of one 100-epoch run: test accuracy is 0 for the first ∼15 epochs, rises sharply between epochs 20 and 45, peaks at 90.0% (epochs 50–70), then drifts to 88.3% while the training loss reaches 0.000 (mild overfitting on 100 puzzles). Fig. 2(b) shows inference-time refinement: accuracy is 86.7% after one supervision step, 88.3% after two, and flat afterwards. Fig. 3 gives a concrete case: after step 1 the model has 3 wrong cells with confidence 0.89, and by step 2 the grid is fully correct (confidence 1.00). This is the self-correction behaviour the paper's Fig. 1 describes.

V. ABLATION

We vary one factor at a time from the default and report mean±std over 3 seeds (Table II, Fig. 4). The paper's columns are taken from its Tables 1 and 3.

Supervision steps. Every supervision step triggers an optimiser update, so with a fixed epoch budget N_{sup}=1 gets 200 updates and N_{sup}=16 gets 3200. This conflates "more supervision steps" with "more training"; the raw runs (0% at $N_{sup}\leq$2) are mostly under-training. We therefore added update-matched controls (epochs scaled as $1600/N_{sup}$, so every run makes 3200 updates; Fig. 5). With updates matched, a single supervision step

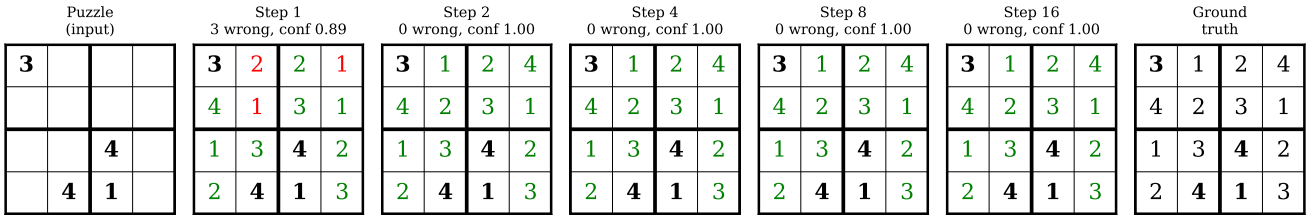


Figure 3. Self-correction on a test puzzle (green = correct, red = wrong, bold = given clue). Left to right: input; predictions after supervision steps 1, 2, 4, 8, 16; ground truth. Step 1 has three errors (confidence 0.89); step 2 repairs all of them.

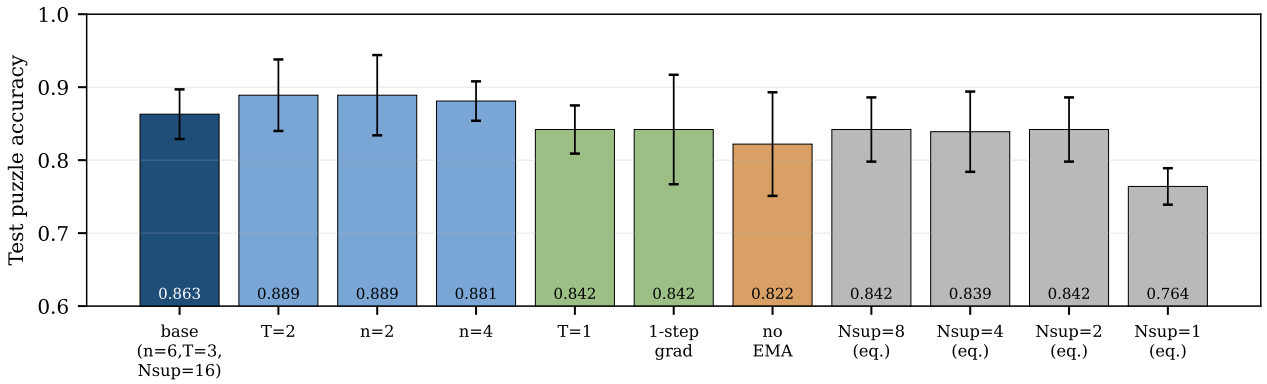


Figure 4. Ablation results (mean ± std, 3 seeds). "(eq.)" = update-matched supervision-step runs. The default is dark blue.

Table II
ABLATION ON 4×4 SUDOKU (OURS) VS. PAPER

Variant	Ours acc (%)	Δ ours	Δ paper
Default (n=6, T=3, N_{sup}=16)	86.3 ± 3.4	—	—
1-step gradient (HRM-style)	84.2 ± 7.5	−2.1	−30.9
No EMA	82.2 ± 7.1	−4.1	−7.5
T=2	88.9 ± 4.9	+2.6	−13.7†
T=1	84.2 ± 3.3	−2.1	−24.2‡
n=4	88.1 ± 2.7	+1.8	—
n=2	88.9 ± 5.5	+2.6	−13.7†
Supervision steps, fixed 100 epochs (fewer updates)			
N_{sup}=1 / 2	0.0/0.0	−86.3	—
N_{sup}=4	27.5 ± 3.8	−58.8	—
N_{sup}=8	66.7 ± 22.1	−19.6	—
Supervision steps, update-matched (3200 updates)			
N_{sup}=1	76.4 ± 2.5	−9.9	—
N_{sup}=2	84.2 ± 4.4	−2.1	—
N_{sup}=4	83.9 ± 5.5	−2.4	—
N_{sup}=8	84.2 ± 4.4	−2.1	—

†paper T=2, n=2 (both reduced). ‡paper T=1, n=1.

Table III
PER-SEED ACCURACY (%), EFFECTIVE DEPTH AND COST

Variant	Depth	Seed 0	Seed 1	Seed 2	Min/run
Default (n=6, T=3)	42	83.2*	85.8	90.0	1.6
T=2	28	83.3	90.8	92.5	1.3
T=1	14	87.5	80.8	84.2	1.0
n=4	30	85.0	90.0	89.2	1.2
n=2	18	82.5	91.7	92.5	0.8
1-step gradient	42	84.2	76.7	91.7	1.1
No EMA	42	87.5	85.0	74.2	1.6

Depth = $T(n+1)\ell$ with ℓ=2. *Restored from a cached run (see Sec. VI).

Table IV
WHICH CLAIMS OF THE PAPER TRANSFER TO OUR SETTING

Claim in paper	Direction in ours	Resolvable?
Tiny 2-layer net with recursion solves the puzzle	yes (86.3%)	yes
Deep supervision is key	yes (N_{sup}=1 worst)	yes, vs. N_{sup}=1 only
EMA stabilises/improves	yes (−4.1 without)	no (within noise)
Full back-prop ≫ 1-step	weak (−2.1)	no
Deeper recursion helps	no	no

is clearly worse (76.4%) than $N_{sup} \geq 2$ (≈84%), supporting the paper's claim that reasoning across steps helps, but there is no measurable gain from 2 to 16 steps; the default's 86.3% is within noise of them.

Gradient, EMA, recursion. Dropping EMA and using the 1-step gradient both lower the mean (by 4.1 and 2.1 points) and greatly increase variance (std 7.1 and 7.5), matching the paper's direction, but the effects are small against a seed std of 3–7 points. The paper's headline result, a 30.9-point drop for the 1-step gradient, does *not* reproduce. Reducing T or n also does not hurt: the T=2, n=2 and n=4 runs are slightly higher than the default (within noise), whereas the paper finds clear losses for shallower recursion.

Summary. Table IV condenses the comparison. The qualitative mechanism (iterative self-correction learned by deep supervision) is reproduced and visible in individual puzzles. The quantitative ablation ordering of the paper is only partly repro-

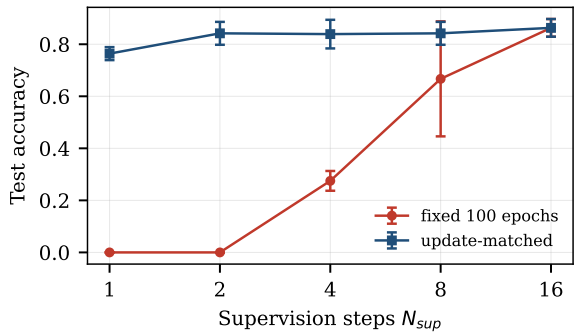


Figure 5. Effect of N_{sup} with a fixed epoch budget (red) vs. matched optimiser updates (blue).

duced: at 16 cells and 288 possible grids the task appears too easy for the extra depth or exact gradients to matter, and the per-run uncertainty ($\approx \pm 3$ points standard error plus 3–7 points across seeds) hides gaps of a few points. Table III also shows that cheaper variants (n=2 at 0.8 min, 18 layers deep) are as accurate as the default at half the cost, which would be a useful finding if it persisted on larger grids.

VI. Failure Cases and What Did Not Work

- **Under-training at low N_{sup}.** With 100 epochs, $N_{sup} \in \{1, 2\}$ scored exactly 0% on all seeds; the model stays on the loss plateau, so this ablation had to be redone update-matched.
- **High variance.** N_{sup}=8 ranged from 42.5% to 85.8% over seeds; no-EMA and 1-step runs ranged 74–92%. With only 120 test puzzles, the 95% binomial interval is about ± 6 points per run, so most single-factor gaps are not statistically resolvable.
- **Unrepaired errors.** In Fig. 2(b) about 14 of 120 test puzzles stay wrong at every step: extra recursion does not fix them, consistent with the model settling into a wrong but self-consistent answer.
- **Overfitting.** Training loss reaches 0.000 while test accuracy falls from 90.0% to 88.3%; EMA and few parameters limit but do not remove this.
- **Claims not reproduced.** The large 1-step-gradient gap and the benefit of deeper recursion. Plausible (untested) reasons: a 4×4 puzzle has 16 cells and few constraints, so shallow recursion already suffices; and our weight decay, learning rate and model width differ from the paper's.
- **Not implemented.** ACT early stopping, data augmentation, the attention variant and the 9×9 benchmark; our scale cannot confirm the paper's 87.4% or its ARC-AGI results. One seed-0 default run was restored from a cached result file and its accuracy (83.2%) is not a multiple of $1/120$, so it may come from a different test draw; re-running it is advisable.

VII. Conclusion

A 268k-parameter, 2-layer recursive network trained on 100 puzzles solves 86% of held-out 4×4 Sudoku and visibly corrects its own errors across supervision steps. At this scale, deep supervision (versus a single step) and EMA help in the direction the paper reports, but full back-propagation and deeper recursion show no resolvable benefit, so the paper's larger effects need harder tasks, more seeds and larger test sets to confirm. Natural next steps are a 9×9 run with augmentation, ACT early stopping, and more seeds.

References

[1] A. Jolicoeur-Martineau, "Less is more: Recursive reasoning with tiny networks," arXiv:2510.04871, 2025.

[2] G. Wang *et al.*, "Hierarchical reasoning model," arXiv:2506.21734, 2025.

[3] ARC Prize Foundation, "The hidden drivers of HRM's performance on ARC-AGI," `https://arcprize.org/blog/hrm-analysis`, 2025.

[4] I. Tolstikhin *et al.*, "MLP-Mixer: An all-MLP architecture for vision," in *Proc. NeurIPS*, 2021.

[5] S. Bai, J. Z. Kolter, and V. Koltun, "Deep equilibrium models," in *Proc. NeurIPS*, 2019.

[6] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," arXiv:1711.05101, 2017.